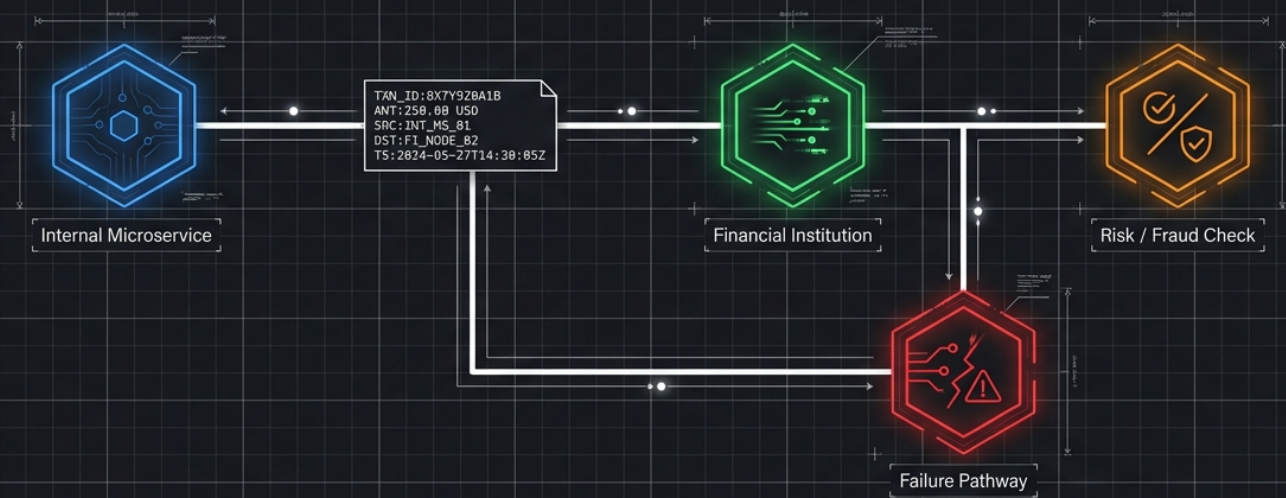
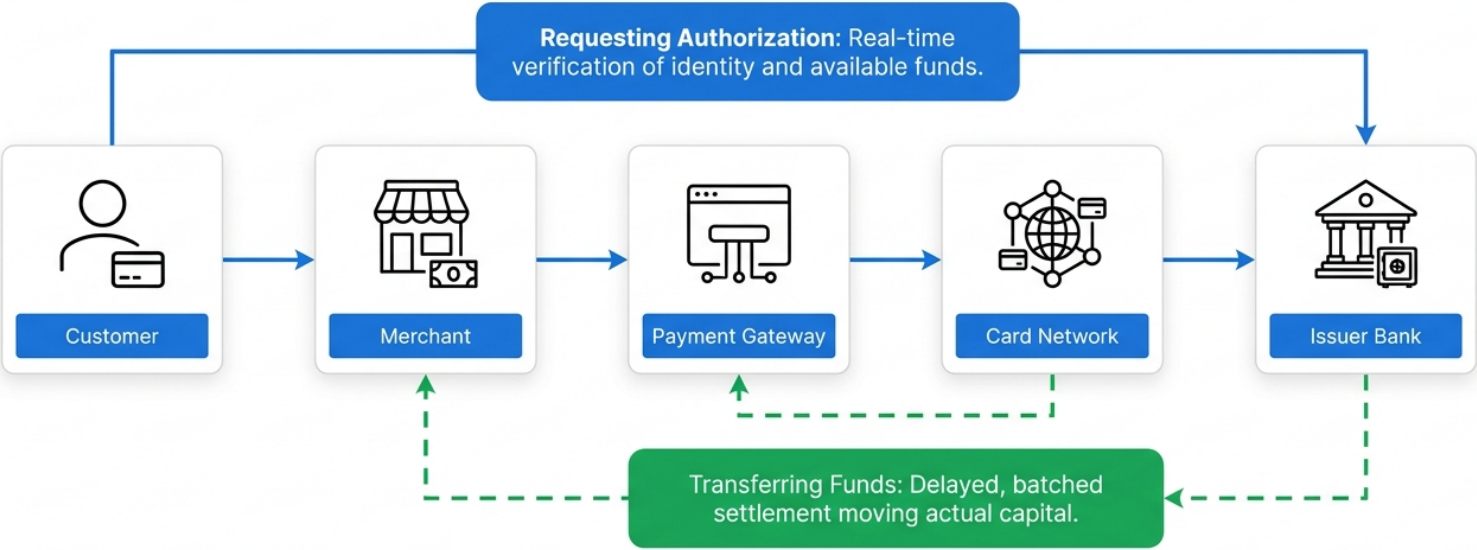


Architecture of a Modern Payment System

A System Design Blueprint: From Network Entities to Distributed Reliability



The Anatomy of a Digital Transaction



A single click of "Pay" triggers a massive, distributed sequence of verifications across multiple independent financial networks.

Network Entities: The Relational Map



Customer

The cardholder initiating the purchase.



Merchant & Store

The entity providing goods/services and the secure collecting payment details.



Payment Gateway

Facilitates and authorizes transactions; requests payment from the customer account.



Issuer Bank

Holds the customer's account; provides the credit card and authorizes funds.



Acquiring Bank

Holds the merchant's account where final funds are deposited.



Cards Network

Visa/Mastercard; validates card info, facilitates the payment, and sets terms.

The Two Phases of a Transaction

Phase 1: Authorization

- 🕒 **Timeline:** Real-time, synchronous, immediate.
- ⚙️ **Mechanism:** Verifies valid payment method, checks for fraud, confirms available credit.
- ✅ **Result:** Approves and reserves the amount on the card, but transfers no actual money.



Milliseconds

Phase 2: Settlement

- 🕒 **Timeline:** Asynchronous, batched, delayed timeframe.
- ⚙️ **Mechanism:** Merchant accumulates authorized transactions and sends a batch to the processor. Processor reconciles for accuracy.
- ✅ **Result:** Finalizes the transaction by actually moving funds from the issuer to the merchant's bank account.



Days

System Design: Defining Requirements

Functional Requirements



Payment Processing: Accept credit/debit cards via a PSP.



User Management: Secure registration, authentication, password reset.



Balance Management: Check account balances (available funds, pending).



Transaction History: View historical dates, amounts, and descriptions.



Mobile Accessibility: Responsive web/mobile interfaces.

Non-Functional Requirements



Integrity & Security: Data encryption, secure storage, zero corruption.



Availability & Reliability: Highly available with failover mechanisms and redundancy.



Scalability & Performance: Seamless scaling for event peaks; low-latency response times for real-time processing.

Resource Estimation: Back of the Napkin Math

INPUT VARIABLES

Peak Volume: 50,000,000 transactions/day
Peak Rate: 5,000,000 txns/hr (~1,400 TPS)

STORAGE CALCULATION

Daily Active Storage = 50M ops × (100B txn + 200B user record)
> **Output:** ~ 5 GB / day

BANDWIDTH ESTIMATION @ 1,400 TPS

Incoming = 1400 TPS × 100 Bytes
> **Output:** ~ 1.12 Mbps
Outgoing = 1400 TPS × 130 Bytes (status codes/tokens)
> **Output:** ~ 1.46 Mbps

SERVER CAPACITY (At Theoretical Peak)

Total RPS / Server Capacity = 50M RPS / 64,000 RPS
> **Output:** ~ 782 servers required

Calculation Complete

Phase 3: Resource Planning

Core API Interfaces

POST /authorizePayment(amount,
card_number, exp_date, CVV, merchant_id)

merchant_id is critical to route the transaction and reserve funds for the correct account.

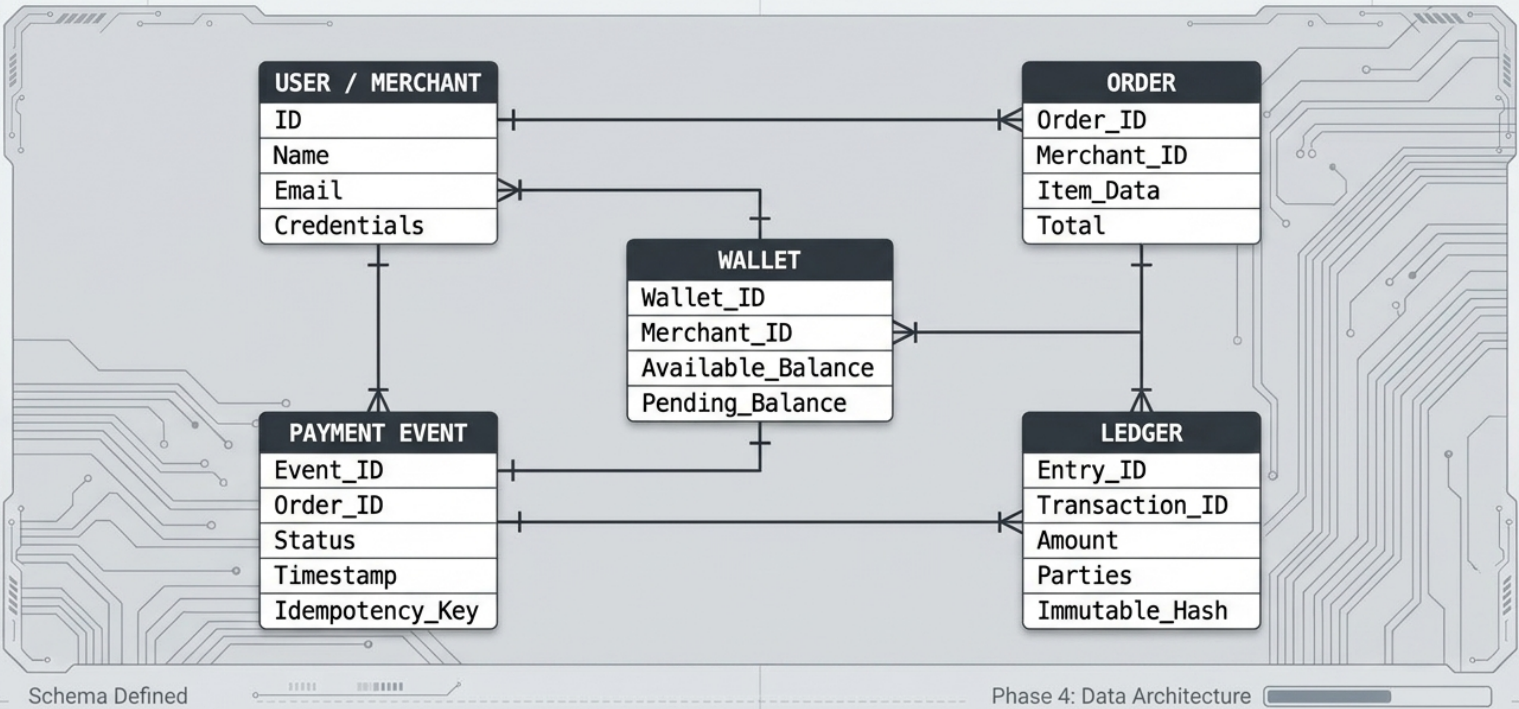
POST /capturePayment(authorization_code,
amount)

Uses the code from Auth to deduct funds and trigger the asynchronous settlement transfer.

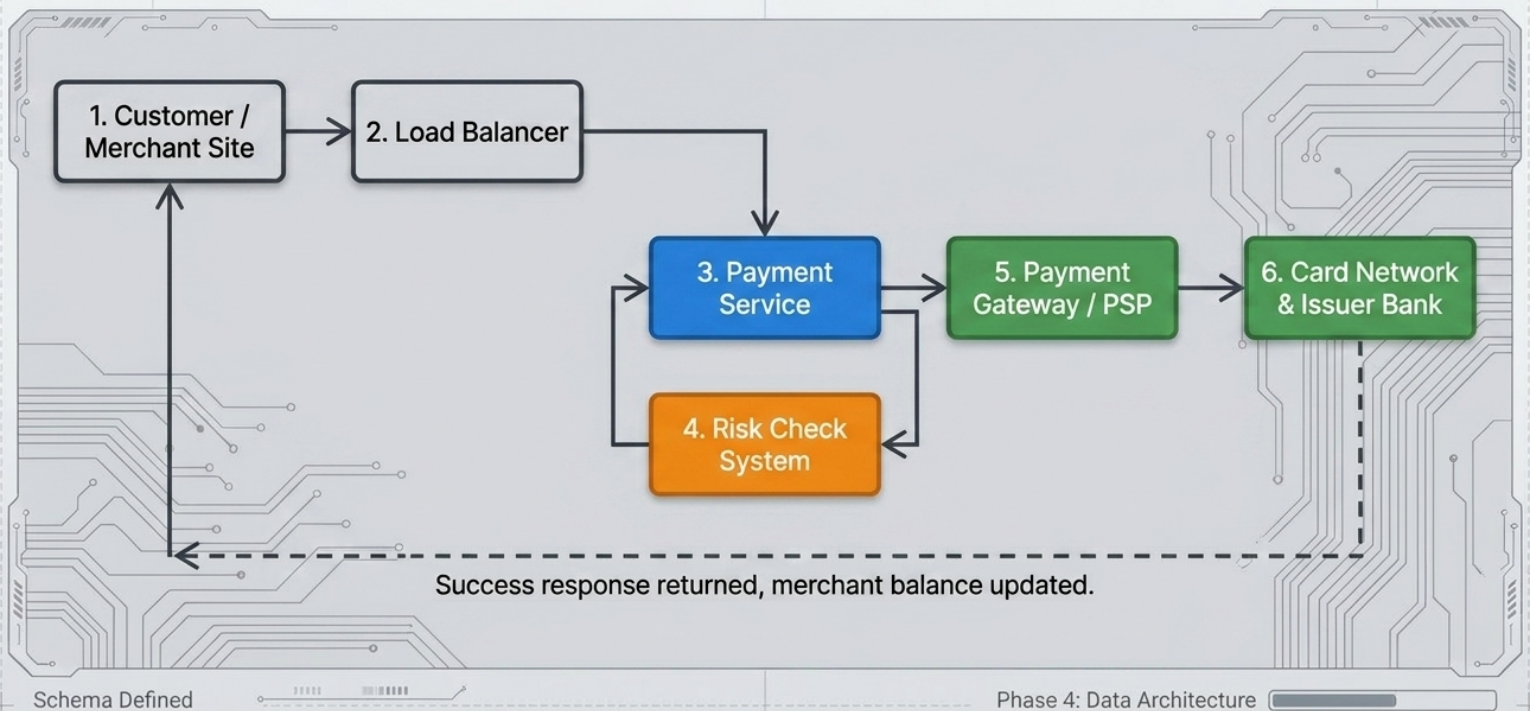
GET /getTransactionHistory(customer_id,
start_date, end_date)

POST /authenticateUser(username, password)

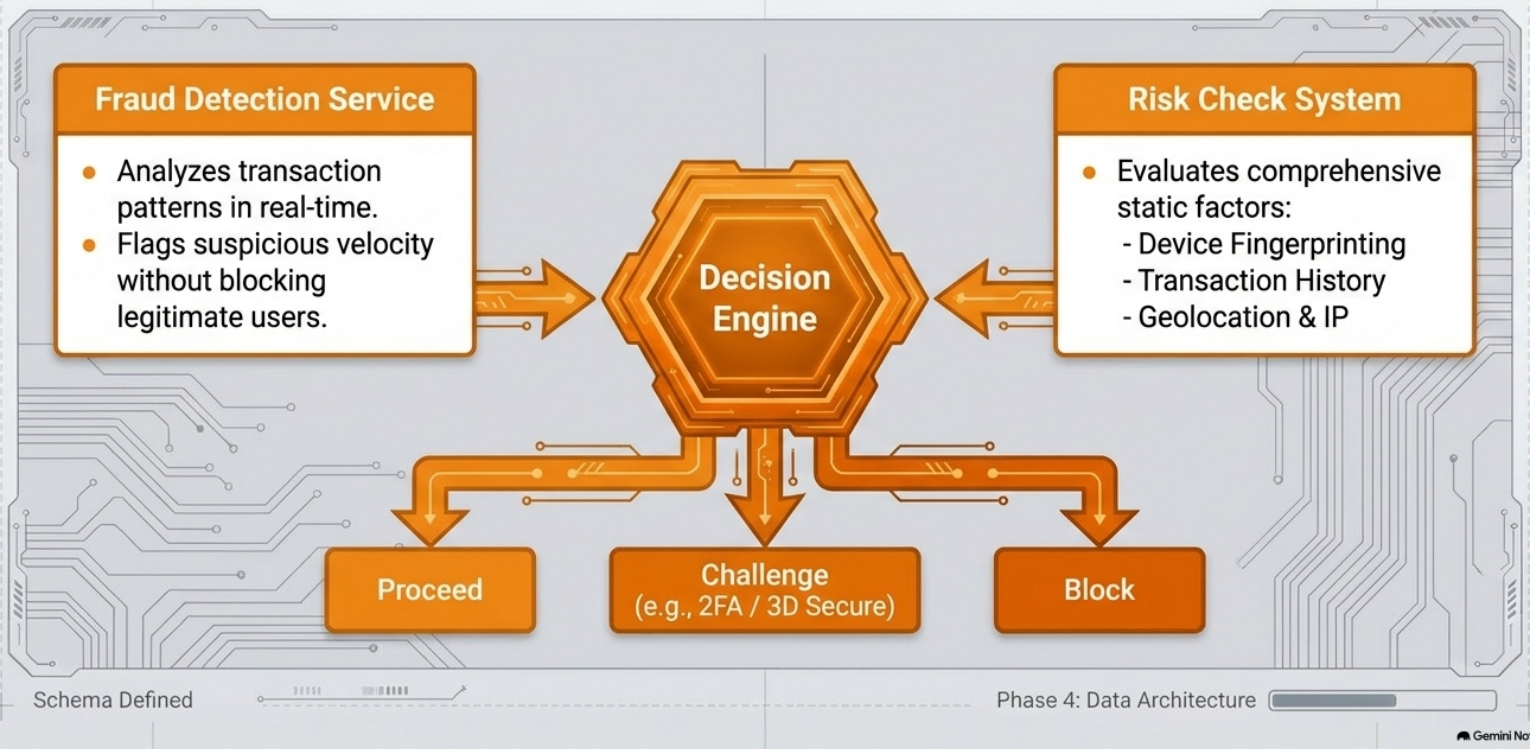
Storage Schema: Managing Financial State



High-Level Architecture: The Happy Path



Decoupling Risk and Fraud Detection



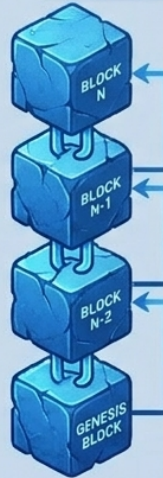
Financial State: The Wallet vs. The Ledger

The Wallet (Current State)



- Maintains up-to-date merchant balances.
- Reflects successful payments and refunds immediately.
- Updated instantly by the Payment Service to ensure fast UI rendering.

The Ledger (Immutable History)

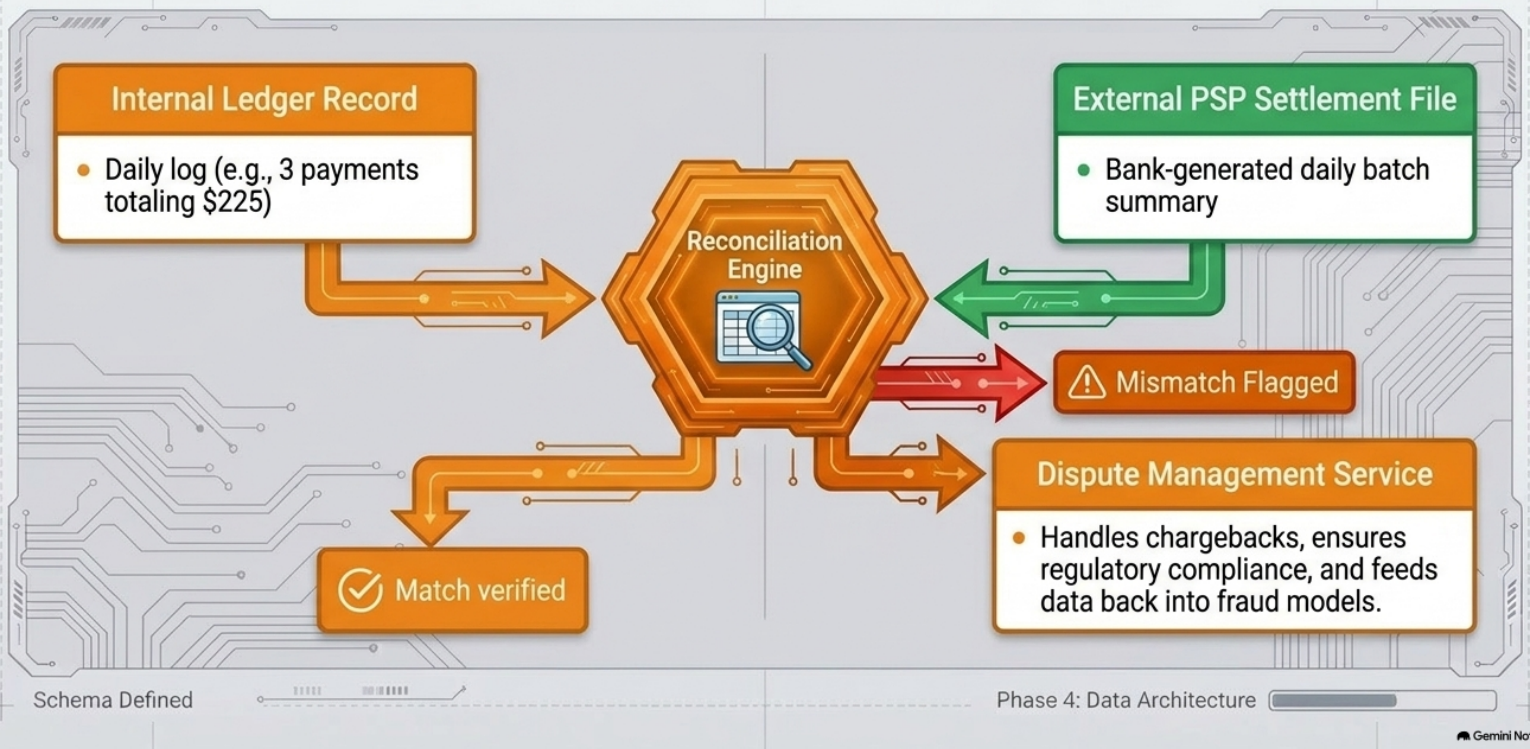


- An append-only database.
- Records every microscopic financial operation.
- Stores transaction ID, amount, parties, status, and timestamp.
- Creates an unalterable, cryptographically secure auditable trail.

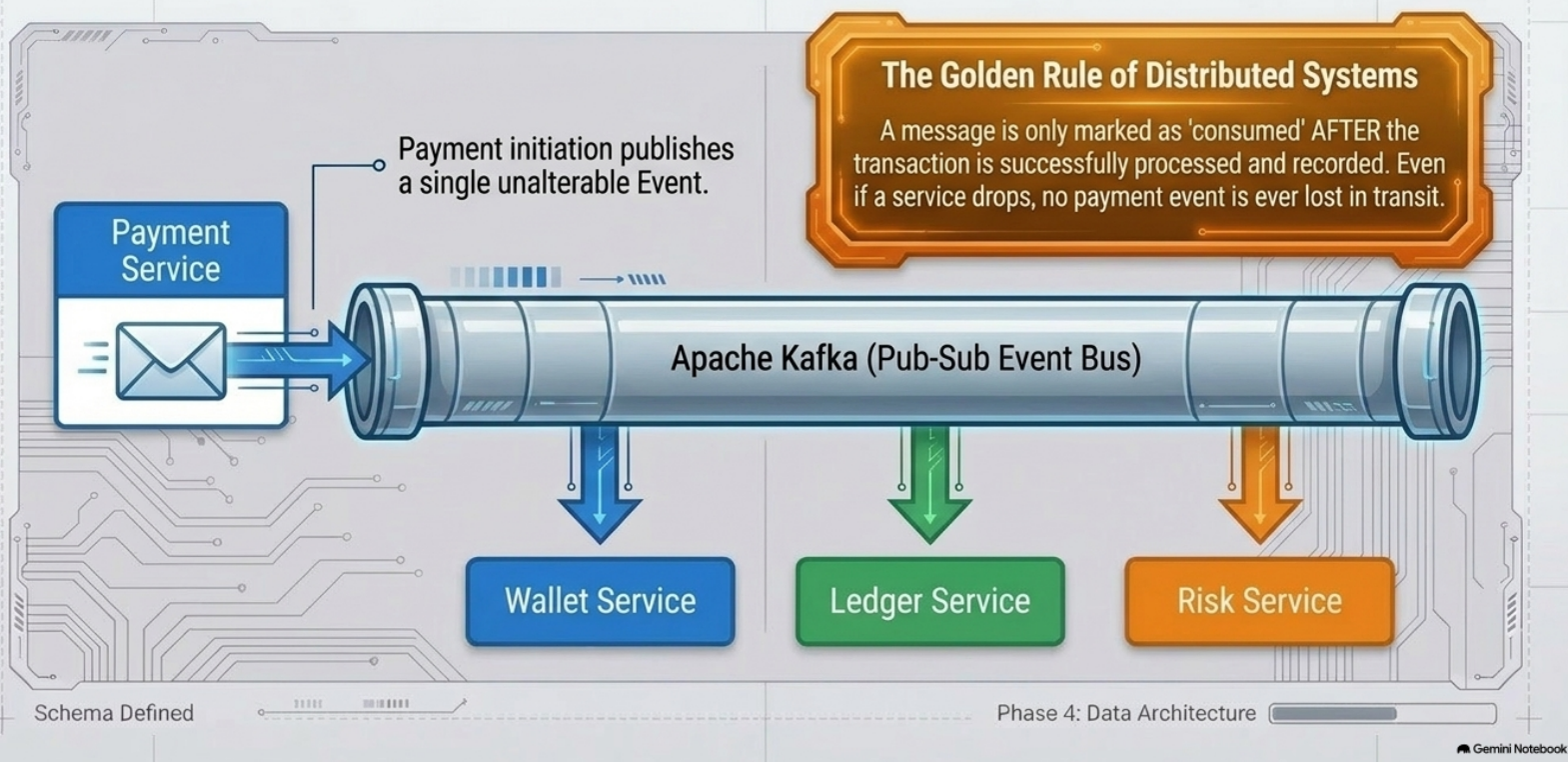
Schema Defined

Phase 4: Data Architecture

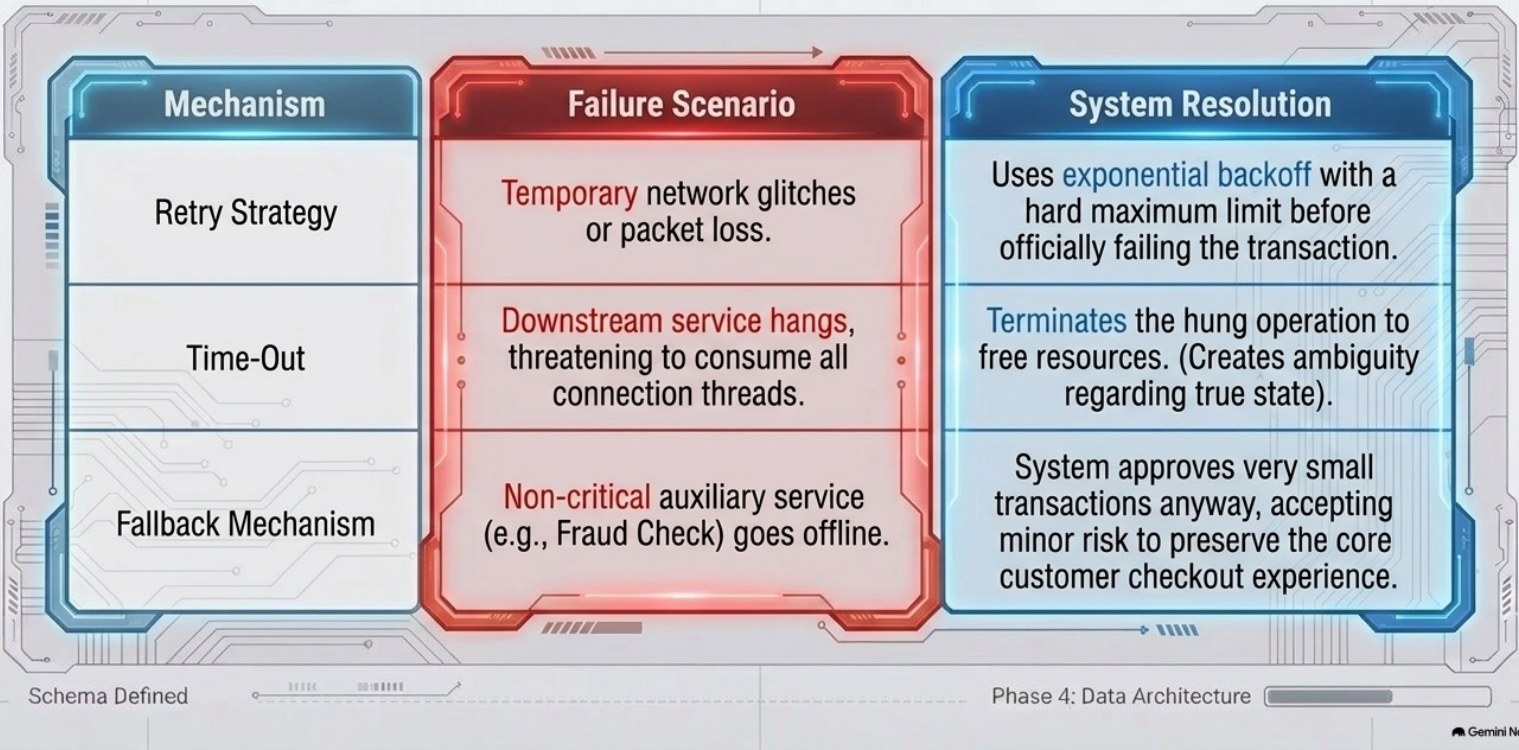
Closing the Loop: Reconciliation & Disputes



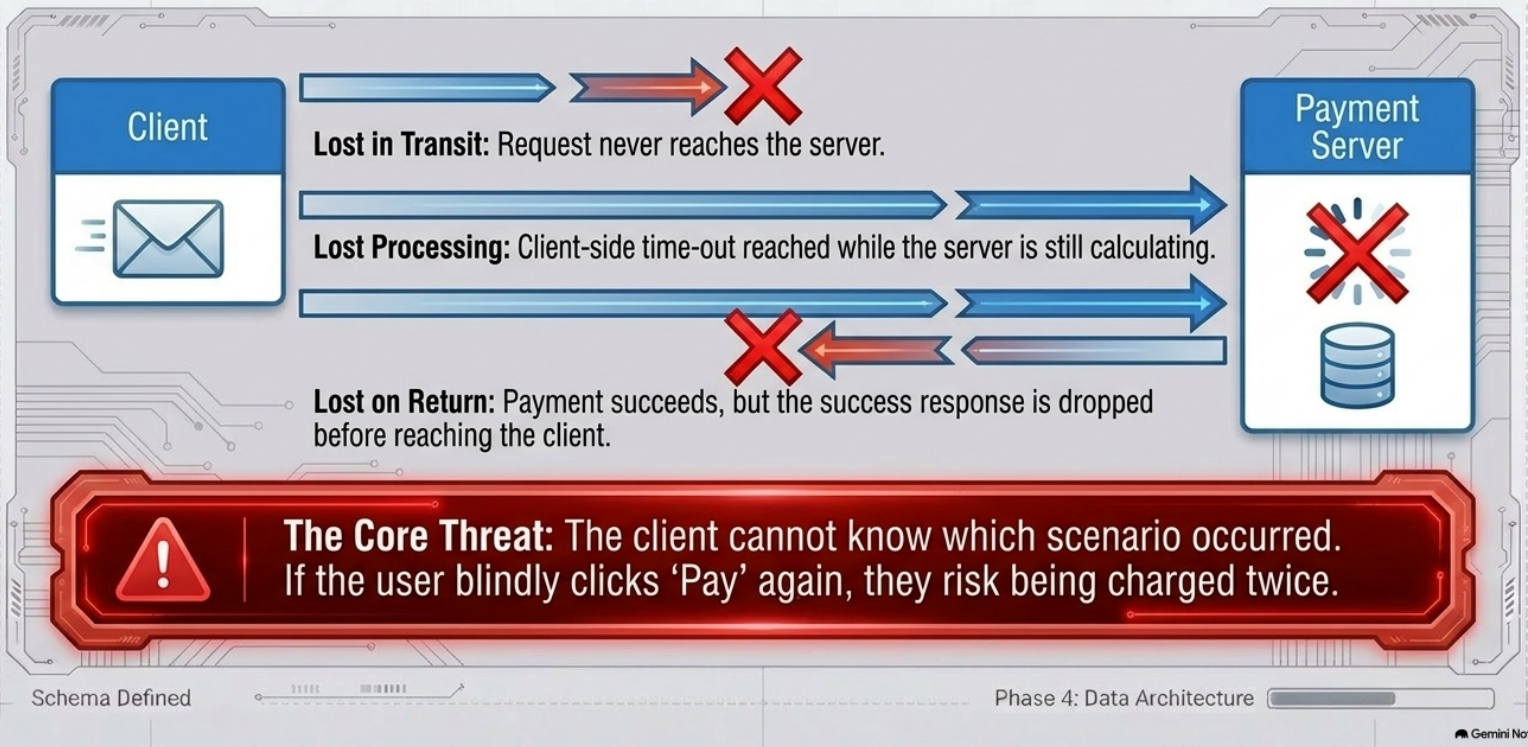
Synthesis: Guaranteeing Completion via Pub-Sub



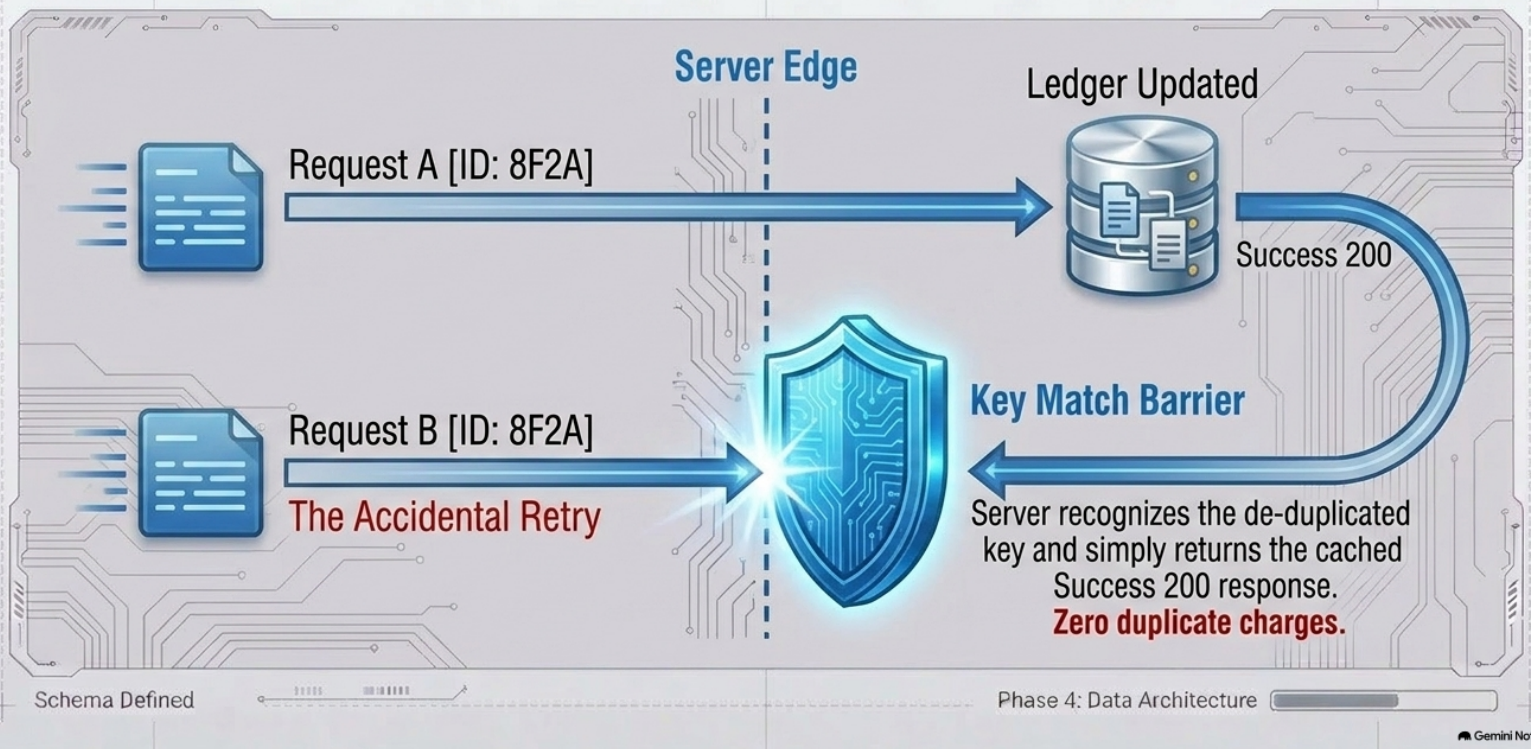
Reliability: Handling Transient Failures



The Danger of Time-Out Ambiguity



Preventing Double Charges: The Idempotency Shield



Validating the Architecture

Functional Fulfillment

- ✓ **Payment Processing** → Coordinated by Payment Service, PSP, and Risk Check.
- ✓ **Transaction History** → Provided by the immutable Ledger log.
- ✓ **Balance Management** → Maintained instantly by the Wallet.

Non-Functional Fulfillment

- ✓ **Scalability** → Achieved by decoupling Risk/Fraud from the core Payment Gateway.
- ✓ **Reliability** → Guaranteed via Kafka Pub/Sub, Idempotency keys, and Fallback processing.

A modern payment system is an exercise in managing distributed state, where **mitigating failure** is just as **critical** as **facilitating success**.